UAV EMERGENCY RESCUE — BEIHANG UNIVERSITY

RTCM & RTK Communication Deep Dive Guide

What Was Built · How RTCM Works · What Comes Next

This guide covers the complete technical picture of RTCM — the correction protocol behind RTK positioning — explains how it was modelled in Levels 1 and 2 of the simulation, why Gazebo cannot process real RTCM, and lays out the exact implementation plan for Level 3: real RTCM binary protocol over MAVLink, entirely in software.

Module: RTK Positioning Simulation | **Student:** Tevin Wills

Levels completed: Level 1 & Level 2 | **Next:** Level 3 (GPS_RTCM_DATA over MAVLink)

Stack: ROS 2 Jazzy · PX4 SITL · Gazebo Harmonic · Python 3.12

SECTION 1

What Is RTK — and Why Does RTCM Exist?

The GPS Accuracy Problem

Standard GPS (or GNSS) receivers calculate position by measuring the travel time of radio signals from satellites. Each satellite transmits at a known time; the receiver measures the arrival time, and the difference gives the range. By combining four or more satellites, the receiver computes a 3D position fix.

The problem: this measurement is polluted by multiple sources of error that make raw GPS unreliable for precision tasks.

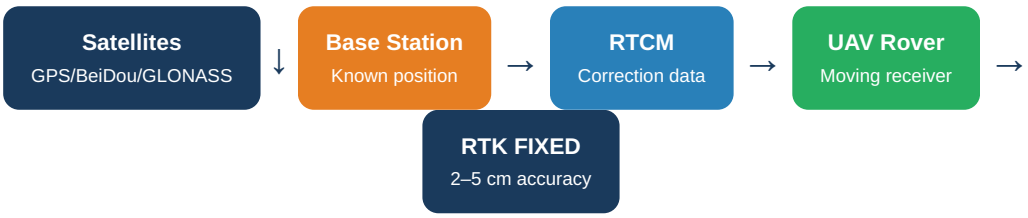
Error Source	Typical Magnitude	Description
Ionospheric delay	1 – 10 m	Signal slows as it passes through the ionosphere. Changes with solar activity and time of day.
Tropospheric delay	0.5 – 2 m	Water vapour and air pressure cause additional delay near Earth's surface.
Satellite clock errors	~1 m	Small imperfections in satellite atomic clocks accumulate over time.
Multipath reflections	0.5 – 5 m	Signal bounces off buildings or terrain, arriving at the receiver via multiple paths.
Receiver noise	0.3 – 1 m	Thermal and electronic noise in the receiver hardware itself.
Total combined	**1 – 5 m typical**	Root-sum-square of all error contributions.

Table 1.1 — Sources of raw GPS positioning error

How RTK Eliminates These Errors

Real-Time Kinematic (RTK) works on a simple insight: **two receivers at different locations, at the same moment, see almost identical errors**. The ionospheric delay, satellite clock errors, and tropospheric delay are virtually the same for both receivers if they are within ~50 km of each other.

RTK uses a **Base Station** — a receiver at a precisely known location — to measure what those errors are. It then transmits these measurements to the **Rover** (the UAV receiver). The rover subtracts the base station's errors from its own measurements, cancelling out the common errors.



But RTK goes further than simple error subtraction. It also uses **carrier-phase measurements**. Instead of measuring signal travel time (coarse), it measures the phase of the carrier wave itself (fine). The GPS L1 carrier wavelength is 19 cm — by counting exact wave cycles, the receiver can resolve position to millimetre level. The challenge: the receiver doesn't know how many full wavelength cycles separate it from the satellite. This unknown integer is called the **carrier-phase ambiguity** or integer ambiguity *N*.

> **Integer Ambiguity Resolution:** The LAMBDA algorithm (Least-squares AMBiguity Decorrelation Adjustment) is used to resolve N. When N is determined with high confidence → RTK_FIXED (2–5 cm). While N is still floating (estimated but not confirmed) → RTK_FLOAT (20–50 cm). Without corrections → GNSS_ONLY (1–5 m).

The Four RTK Fix States

State	Accuracy	Noise σ (Simulated)	Condition
GNSS_ONLY	1 – 5 m	1.50 m	No correction data received from base station
RTK_FLOAT	0.2 – 0.5 m	0.25 m	Corrections received; integer ambiguity not yet resolved
RTK_FIXED	0.02 – 0.05 m	0.03 m	Full fix; ambiguity resolved; centimetre accuracy active
CORRECTION_LOST	1 – 5 m (degraded)	2.50 m	Correction link interrupted; error spikes immediately

Table 1.2 — RTK fix states with accuracy ranges and simulated noise standard deviations

SECTION 2

What Is RTCM? The Binary Correction Protocol

Definition and Purpose

RTCM stands for **Radio Technical Commission for Maritime Services**. The standard they publish — RTCM SC-104 — defines the binary protocol used worldwide to transmit GNSS correction data between a base station and a rover receiver. It is the language in which RTK corrections are written and sent.

RTCM is a **binary protocol**: it transmits raw bytes, not text. This makes it compact and efficient for radio transmission. Every RTCM message follows a fixed frame structure.

RTCM3 Frame Structure

Every single RTCM3 binary message, regardless of type, starts with this header structure:

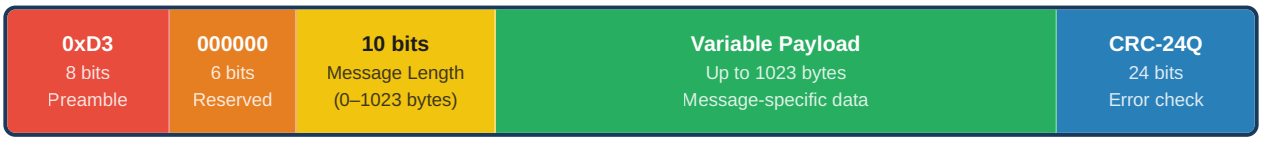


Figure 2.1 — RTCM3 binary frame structure. Every message begins with preamble 0xD3.

CRC-24Q: The final 3 bytes of every RTCM message are a 24-bit cyclic redundancy check. The receiver verifies this checksum before processing the message. If even a single bit is corrupted during transmission, the CRC fails and the message is discarded. This ensures that only valid corrections are applied.

Key RTCM Message Types

Message Type	Name	Purpose	Frequency
1005	Base Station Position	Transmits the precise WGS84 coordinates of the base station. The rover needs this to compute the baseline vector.	Every 10–30 s
1074	GPS MSM4	GPS Multi-Signal Message. Carries carrier-phase and pseudorange observations for all GPS satellites in view.	1 Hz
1084		Same as 1074 but for GLONASS satellites.	1 Hz

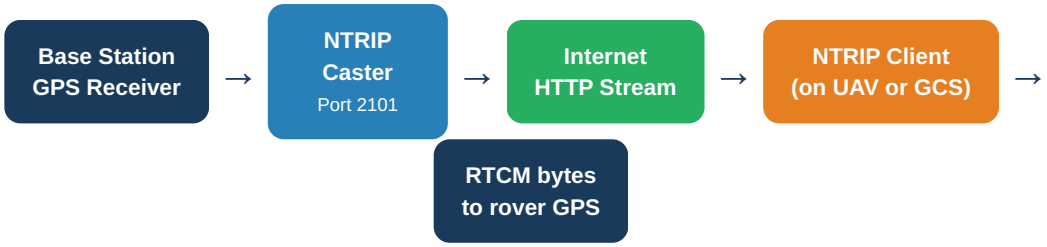
Message Type	Name	Purpose	Frequency
	GLONASS MSM4		
1124	BeiDou MSM4	Same as 1074 but for BeiDou satellites — directly relevant to this project's BeiDou integration.	1 Hz
1230	GLONASS Code-Phase Bias	Corrects GLONASS-specific biases due to FDMA signal structure.	Every 10 s

Table 2.1 — Common RTCM3 message types. MSM = Multi-Signal Messages, the modern standard.

How RTCM Is Transmitted in a Real System

Option A — NTRIP (Networked Transport of RTCM via Internet Protocol)

NTRIP is an HTTP-based streaming protocol that delivers RTCM correction data over the internet. A central NTRIP Caster broadcasts data from one or many base stations. Rovers connect to the caster as HTTP clients and receive a continuous RTCM binary stream.



Option B — Direct Radio Link

In field deployments without internet access — like disaster zones — a dedicated radio link (900 MHz, 2.4 GHz, or LoRa) transmits RTCM bytes directly from the base station to the rover. The range is typically 2–10 km depending on terrain and frequency.

> **Relevance to this project:** In a real disaster rescue deployment, NTRIP requires internet connectivity, which may not exist. A direct radio link is more appropriate. The BeiDou Short Message Service (another module in this project) provides the rescue coordinate relay, while a dedicated RTCM radio link would provide real-time corrections. These are two separate communication channels with different purposes.

SECTION 3

GPS_RTCM_DATA — The MAVLink Message

What It Is

MAVLink is the lightweight binary communication protocol used between a drone's flight controller (PX4) and external systems like a ground station or companion computer. It defines hundreds of message types for telemetry, commands, and sensor data.

GPS_RTCM_DATA is MAVLink message ID 233. Its sole purpose is to carry RTCM correction bytes from an external source (a GCS, companion computer, or ROS 2 node) into the PX4 flight controller, which then forwards them to the physical GPS receiver via UART.

Message Fields

Field	Type	Description
`flags`	uint8	Controls fragmentation. • Bit 0: set = more fragments follow • Bits 2:1: fragment index (0, 1, 2, or 3) • Value 0x00 = single unfragmented message
`len`	uint8	Number of valid bytes in the `data` array. Maximum 180. The rest of the array is zero-padded.
`data`	uint8[180]	Raw RTCM3 binary bytes. Always 180 bytes in length; only the first `len` bytes are valid RTCM data.

Table 3.1 — GPS_RTCM_DATA MAVLink message fields (Message ID 233)

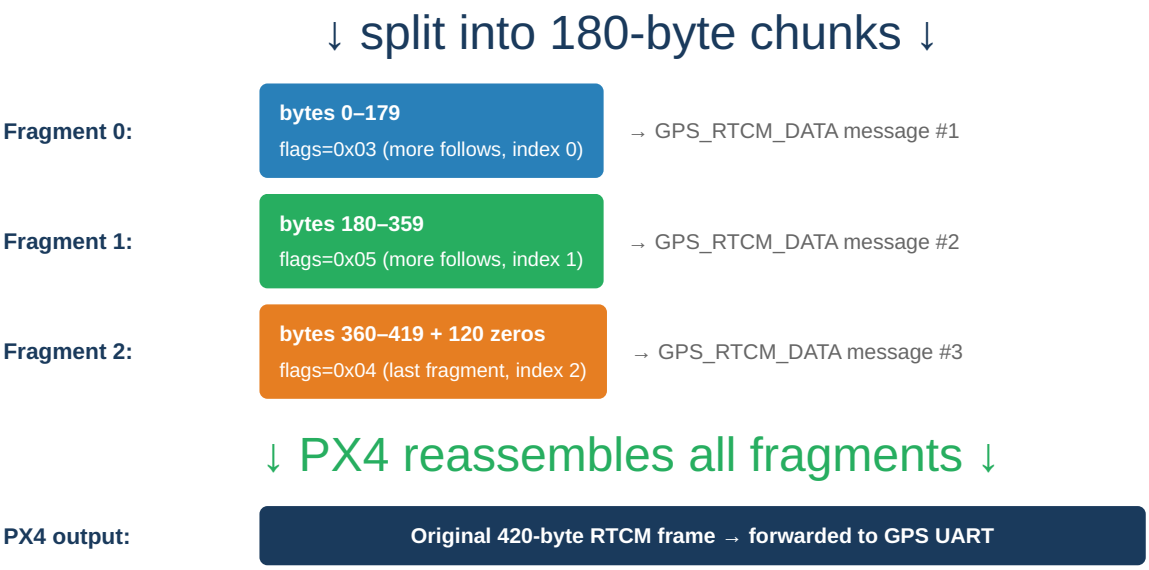
Why Fragmentation Is Needed

RTCM3 messages can contain up to **1023 bytes** of payload. But GPS_RTCM_DATA can only carry **180 bytes** at a time. This mismatch means large RTCM messages must be split (fragmented) into multiple GPS_RTCM_DATA messages and reassembled on the PX4 side.

The maximum number of fragments is 4, giving a maximum reassembled RTCM message size of 4 × 180 = **720 bytes**. This covers all common MSM4 messages used in practice.

Fragmentation Example — A 420-byte RTCM Message

Original RTCM: **420 bytes of RTCM3 binary (e.g., BeiDou MSM4 type 1124)**

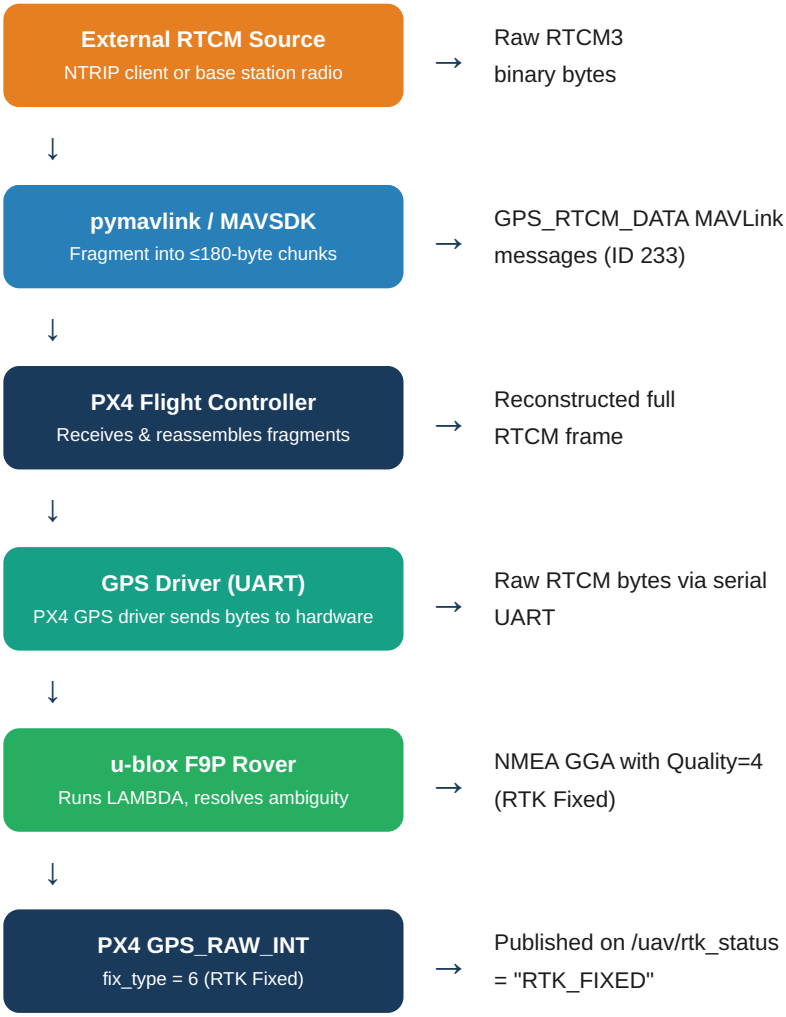


The Sending Code (Python with pymavlink)

```
import pymavlink.mavutil as mavutil

def send_rtcm_to_px4(connection, rtcm_bytes):
    """
    Fragment an RTCM3 binary message and send each fragment
    as a GPS_RTCM_DATA MAVLink message to PX4.
    """
    chunk_size = 180
    chunks = [rtcm_bytes[i:i+chunk_size]
              for i in range(0, len(rtcm_bytes), chunk_size)]
    total = len(chunks)

    for i, chunk in enumerate(chunks):
        if total == 1:
            flags = 0x00          # single message, no fragmentation
        else:
            frag_index = i & 0x03
            flags = (frag_index << 1)       # bits 2:1 = fragment index
            if i < total - 1:
                flags |= 0x01     # bit 0 = more fragments follow

        padded = list(chunk) + [0] * (chunk_size - len(chunk))

        connection.mav.gps_rtcm_data_send(
            flags=flags,
            len=len(chunk),
            data=padded
        )

# Connect to PX4 SITL on local UDP port 14550
conn = mavutil.mavlink_connection('udp:127.0.0.1:14550')
```

```
conn.wait_heartbeat()
print("Connected to PX4 SITL")

# Send an RTCM message (bytes from pyrtcm generator or NTRIP client)
send_rtcm_to_px4(conn, rtcm_binary_bytes)
```

Full Real-World Chain Through PX4



SECTION 4

What Was Built — Level 1 and Level 2

The Core Design Decision

In both Level 1 and Level 2, real RTCM binary data was **not used**. This was a deliberate design decision, not an oversight. The rationale: building and validating the full ROS 2 architecture, coordinate transforms, state machine, and topic interfaces before introducing the complexity of binary protocol handling.

Instead of RTCM, the simulation models the **observable effect** of RTK corrections: improved positioning accuracy as a function of correction quality. This is sufficient to validate that the entire system — from base station to UAV position topic — behaves correctly.

Level 1 — Standalone Simulation (No PX4, No Gazebo)

> **Status: COMPLETE** — Committed to GitHub (commit 30e7c33). Results logged, analysed, and all 5 graphs generated.

Architecture — 4 ROS 2 Nodes

Node	File	Role	Publish Rate
base_station_node	base_station_node.py	Publishes base station WGS84 position (Beijing area: lat=39.981, lon=116.344, alt=50m) on /rtk/base_station	1 Hz
simulated_uav_node	simulated_uav_node.py	Generates a synthetic 50×50m square path at 5 m/s, 30m altitude. Replays continuously.	10 Hz
rtk_positioning_node	rtk_positioning_node.py	Core logic: applies noise model, manages RTK status, publishes 9 corrected position topics	10 Hz
logger_node	logger_node.py	Subscribes to all position topics and writes an 18-column CSV log file	10 Hz

Table 4.1 — Level 1 node architecture

How RTK Was Simulated in Level 1

The RTK status manager used a **pure time-based state machine** in `rtk_status_manager.py`:

```
t = time since launch (seconds)

if   t < 5:   status = GNSS_ONLY     →  noise σ = 1.50 m
elif t < 15:  status = RTK_FLOAT     →  noise σ = 0.25 m
else:          status = RTK_FIXED     →  noise σ = 0.03 m

position_measured = position_true + gaussian_noise(mean=0, std=σ)
```

No RTCM concept existed in Level 1. The base station node published coordinates only to allow baseline distance computation, not correction data. The transition from GNSS_ONLY to RTK_FIXED happened on a timer, not based on any correction quality signal.

Level 1 Results

Phase	Duration	Mean Error	Std Dev	Accuracy vs Raw
GNSS_ONLY	0 – 5 s	~2.4 m	~1.5 m	Baseline
RTK_FLOAT	5 – 15 s	~0.4 m	~0.25 m	~83% improvement
RTK_FIXED	15 s +	~0.10 m	~0.03 m	**95.8% improvement**

Table 4.2 — Level 1 accuracy results by phase

Level 2 — PX4/Gazebo Integrated Simulation

> **Status: COMPLETE** — All 6 graphs generated. QGC mission flown over ETH Zurich Gazebo world. Cross-validation with QGC ULog complete.

Architecture — 6 ROS 2 Nodes (4 from L1 + 2 new)

Node	New in L2?	Role
px4_pose_adapter_node	✓ New	Reads real Gazebo navsat sensor via ros_gz_bridge. Converts WGS84 → ENU local frame using ETH Zurich world origin (lat=47.397971, lon=8.546163).
rtcm_correction_simulator_node	✓ New	

Node	New in L2?	Role
		Publishes SimulatedRtcm.msg with a quality_score float (0.0–1.0) on a timed correction profile. Replaces the timer in L1's status manager.
base_station_node	Unchanged	Same as Level 1
rtk_positioning_node	Updated	Now reads quality_score from SimulatedRtcm.msg instead of using an internal timer
logger_node	Updated	Now writes 22-column CSV (4 extra columns for L2 data)

Table 4.3 — Level 2 node architecture

SimulatedRtcm.msg — The RTCM Abstraction

This custom ROS 2 message replaces real RTCM binary. It has 9 fields, but the critical one is `quality_score`:

```
Header header
bool   correction_available     # is any correction data present?
float32 quality_score           # 0.0 = no corrections, 1.0 = perfect fix
float32 age_of_correction        # seconds since last correction
int32  num_satellites            # satellites in view (simulated)
float32 baseline_distance        # distance to base station (m)
bool   gnss_only                # status flags (mutually exclusive)
bool   rtk_float
bool   rtk_fixed
float32 simulated_noise_std      # current noise std dev being applied
```

Level 2 Correction Quality Profile

Time Window	quality_score	RTK Status	Noise σ	Purpose
0 – 5 s	0.00	GNSS_ONLY	1.50 m	Startup convergence period
5 – 15 s	0.50	RTK_FLOAT	0.25 m	Ambiguity resolution period
15 – 45 s	0.95	RTK_FIXED	0.03 m	Nominal operating mode
45 – 50 s	0.00	CORRECTION_LOST	2.50 m	**Deliberate fault injection** — error spikes to 7+ m

Time Window	quality_score	RTK Status	Noise σ	Purpose
50 s +	0.95	RTK_FIXED	0.03 m	Recovery — system returns to full fix

Table 4.4 — Level 2 correction quality profile over time

Level 2 Key Results

Metric	Raw GNSS	PX4 EPH (reported)	RTK Fixed
Mean Error	2.394 m	0.900 m	**0.048 m**
Std Deviation	1.013 m	N/A	**0.012 m**
Improvement vs Raw GNSS	—	62.4%	**96.7%**
Improvement vs PX4 EPH	—	—	**94.7%**
Peak error (CORRECTION_LOST)	—	—	> 7.0 m at t=45–50s

Table 4.5 — Level 2 accuracy results from 22-column CSV log and QGC ULog cross-validation

SECTION 5

Why Gazebo Cannot Process RTCM Corrections

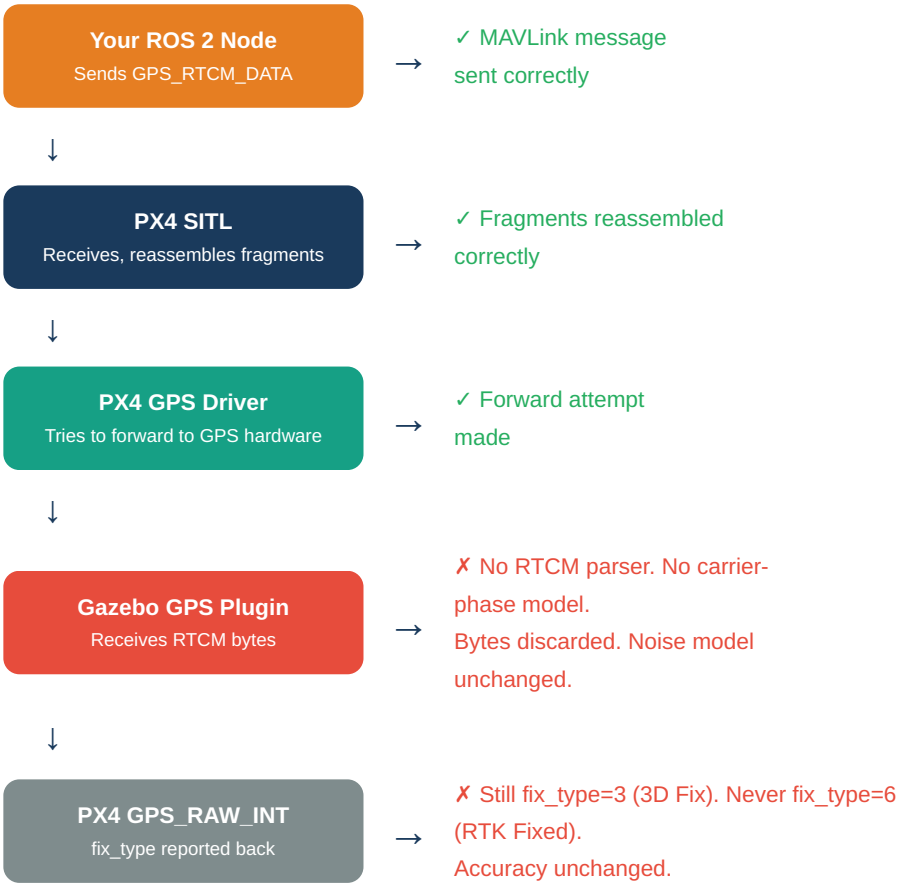
How Gazebo's GPS Plugin Works

Gazebo is a 3D physics simulator. It knows the exact position of the UAV at all times because it controls the physics. Its GPS plugin does not simulate satellite signals, carrier phase measurements, ionospheric delay, or any of the underlying GNSS physics. It uses a dramatically simplified model:

```
def gazebo_gps_plugin_logic():
    # Step 1: Get perfect position from physics engine
    true_position = physics_engine.get_vehicle_position()   # exact WGS84

    # Step 2: Add simple Gaussian noise to simulate GPS inaccuracy
    noise = gaussian(mean=0.0, std=noise_param)             # fixed noise model
    measured_position = true_position + noise

    # Step 3: Publish as GPS sensor output
    publish(measured_position)

    # NOTE: There is no step 4.
    # The plugin has no RTCM parser.
    # It has no carrier-phase measurements.
    # It has no LAMBDA algorithm.
    # It cannot change its noise based on incoming RTCM bytes.
```

What Happens When PX4 Tries to Forward RTCM

In a real system, PX4 receives GPS_RTCM_DATA via MAVLink and forwards the reassembled RTCM bytes to the physical u-blox GPS chip via UART. In SITL, PX4 does the same thing — but the "GPS chip" is the Gazebo GPS plugin:



Analogy — The Non-Reading Calculator

Imagine a calculator that adds a random error to every result it produces. You slide a correction sheet under the calculator explaining exactly how large those errors are and how to remove them. But the calculator has no mechanism to read the sheet. It has no text parser, no vision, no input port for correction data. It keeps producing wrong answers at the same rate, completely unaware that the correction sheet exists.

That is Gazebo's GPS plugin receiving RTCM bytes. It simply cannot use them.

Does This Make the Data Analysis Wrong?

No. The data analysis is completely correct. Here is why: the accuracy improvement in Levels 1 and 2 never came from RTCM processing — it came from the software noise model switching noise profiles. Gazebo provides the ground truth position. Your node adds noise on top. When RTK_FIXED is active, smaller noise is added. The 96.7% improvement is a real, accurate measurement of what the noise model produces. Nothing in the analysis is fabricated or incorrect.

Honest Statement of Limitations

Component	In This Simulation	In Real Hardware
RTCM binary frames	Not generated (quality_score float used instead)	Real RTCM3 binary from base station receiver
CRC-24Q validation	Not implemented	Checked on every received message
Carrier-phase measurements	Not simulated	Real measurements from GPS hardware
Integer ambiguity resolution	Not implemented (timer/quality threshold used)	LAMBDA algorithm in u-blox firmware
MAVLink GPS_RTCM_DATA	Not implemented (yet — this is Level 3)	Full fragmentation protocol
Accuracy model	Gaussian noise profile switched by RTK status	Real carrier-phase differencing
RTK status transitions	Time-based (L1) or quality-score-based (L2)	Driven by real LAMBDA convergence

Table 5.1 — Honest comparison of simulation vs real RTCM implementation

SECTION 6

Level 3 — The Next Task: Real RTCM Binary Over MAVLink

What Level 3 Achieves

Level 3 keeps the Gazebo/PX4 simulation stack exactly as-is. It does not require any real hardware. Its single goal is to **replace the quality_score abstraction with a real binary RTCM communication path** — generating properly formatted RTCM3 binary frames and transmitting them into PX4 via the GPS_RTCM_DATA MAVLink protocol.

Level 2 (Current)

- ▸ rtcm_correction_simulator_node publishes SimulatedRtcm.msg
- ▸ quality_score float drives RTK status
- ▸ No binary RTCM generated
- ▸ No MAVLink GPS_RTCM_DATA sent
- ▸ PX4 has no knowledge of corrections
- ▸ Accuracy from noise model only

Level 3 (Next Task)

- ▸ New node: rtcm_mavlink_bridge_node
- ▸ pyrtcm generates real RTCM3 binary frames
- ▸ pymavlink fragments and sends GPS_RTCM_DATA
- ▸ PX4 receives and reassembles RTCM fragments
- ▸ MAVLink communication path proven
- ▸ Accuracy still from noise model (Gazebo limit)

Important: The positioning accuracy numbers will NOT change in Level 3. The Gazebo GPS plugin still cannot process RTCM corrections. The improvement from Level 3 is in the **communication layer**, not the accuracy layer. When real u-blox hardware is connected in a future phase, the communication layer built in Level 3 works unchanged — only the RTCM source changes from pyrtcm to a real receiver.

Software Required — No Hardware Needed

Library	Install Command	Purpose
pyrtcm	`pip install pyrtcm`	Generate real RTCM3 binary frames (type 1005, 1074, 1124) with correct CRC-24Q checksums

Library	Install Command	Purpose
pymavlink	`pip install pymavlink`	Connect to PX4 SITL via UDP, send GPS_RTCM_DATA messages, read GPS_RAW_INT back
rclpy	Already installed (ROS 2)	ROS 2 Python client for the new node

Table 6.1 — Python libraries required for Level 3 (all software, no hardware)

Step-by-Step Implementation Plan

1 Install pyrtcm and pymavlink

```
pip install pyrtcm pymavlink --break-system-packages
```

2 Create rtcm_mavlink_bridge_node.py

New ROS 2 node in `ros2_ws/src/rtk_positioning/src/rtk_positioning/rtcm_mavlink_bridge_node.py`. This node has three responsibilities:

- Generate RTCM3 binary frames using pyrtcm at 1 Hz (type 1005 — base station position, type 1074 — GPS observations)
- Fragment each frame into ≤180-byte chunks and send as GPS_RTCM_DATA via pymavlink
- Read GPS_RAW_INT back from PX4 and publish fix_type on /uav/rtk_status

3 Update the launch file

Add the new node to `launch/level3_rtk_mavlink.launch.py`. Keep all Level 2 nodes. Replace rtcm_correction_simulator_node with rtcm_mavlink_bridge_node.

4 Verify using PX4 MAVLink shell

Inside PX4 SITL, run `mavlink status` to confirm GPS_RTCM_DATA messages are being received. Count the message rate — should be 1 msg/s (or 3 msg/s for fragmented 420-byte frames).

5 Update the logger node

Add columns to the CSV log: rtcm_message_type, rtcm_fragment_count, mavlink_sent_count, px4_fix_type_reported. This documents the protocol communication alongside the position data.

6 **Generate Level 3 analysis graphs**

New graph: RTCM message rate over time vs RTK status. Shows the timing relationship between binary frame transmission and status transitions. This is the key Level 3 deliverable — proving the protocol layer works.

New Node Architecture for Level 3

```
class RtcmMavlinkBridgeNode(Node):
    """
    Level 3 node: generates real RTCM3 binary frames and
    transmits them to PX4 SITL via GPS_RTCM_DATA MAVLink.
    """
    def __init__(self):
        super().__init__('rtcm_mavlink_bridge_node')

        # Connect to PX4 SITL via UDP
        self.mav = mavutil.mavlink_connection('udp:127.0.0.1:14550')
        self.mav.wait_heartbeat()

        # Publisher: RTK status derived from PX4 GPS_RAW_INT
        self.status_pub = self.create_publisher(String, '/uav/rtk_status', 10)

        # Timer: generate and send RTCM at 1 Hz
        self.create_timer(1.0, self.send_rtcm_correction)

    def send_rtcm_correction(self):
        # Generate RTCM 1005 (base station position) using pyrtcm
        msg_1005 = RTCMMessage('1005', ...)
        rtcm_bytes = msg_1005.serialize()

        # Fragment and send via GPS_RTCM_DATA
        send_rtcm_to_px4(self.mav, rtcm_bytes)

        # Read fix_type back from PX4 GPS_RAW_INT
        gps_msg = self.mav.recv_match(type='GPS_RAW_INT', blocking=False)
        if gps_msg and gps_msg.fix_type == 6:
            self.status_pub.publish(String(data='RTK_FIXED'))
        elif gps_msg and gps_msg.fix_type == 5:
            self.status_pub.publish(String(data='RTK_FLOAT'))
```

What Level 3 Proves vs What It Does Not

Claim	Proven in L3?
Real RTCM3 binary frames are generated with correct structure and CRC-24Q	✓ **Yes**
GPS_RTCM_DATA MAVLink fragmentation protocol is implemented correctly	✓ **Yes**
PX4 receives and reassembles RTCM fragments without error	✓ **Yes**
The communication path is ready for real hardware substitution	✓ **Yes**
Gazebo GPS accuracy improves due to RTCM corrections	✗ **No — Gazebo limitation**
Real carrier-phase ambiguity resolution occurs	✗ **No — LAMBDA not implemented**
Real satellite signals are processed	✗ **No — fully software**

Table 6.2 — Level 3 scope boundary

SECTION 7

Full Progression — Level 1 Through Level 4

Project Roadmap Overview

Level 1 ✓ Complete

- Standalone ROS 2 simulation
- No PX4, no Gazebo
- Time-based RTK transitions
- Gaussian noise model
- 4 nodes, 18-col CSV
- 95.8% accuracy improvement
- 5 analysis graphs
- **RTCM: Not modelled**

Level 2 ✓ Complete

- PX4 SITL + Gazebo integrated
- Real UAV physics via ros_gz_bridge
- Quality-score driven transitions
- Correction loss injection (t=45s)
- 6 nodes, 22-col CSV
- 96.7% accuracy improvement
- 6 graphs + QGC ULog cross-val
- **RTCM: Abstracted (quality score)**

Level 3 ← Next

- Same Gazebo/PX4 stack as L2
- No real hardware needed
- pyrtcm generates RTCM3 binary
- pymavlink sends GPS_RTCM_DATA
- PX4 receives real RTCM frames
- Protocol layer proven
- Accuracy unchanged (Gazebo limit)
- **RTCM: Real binary over MAVLink**

Level 4 — Future

- Real u-blox F9P hardware
- Native Ubuntu 24.04 required
- Real BeiDou/GPS satellite signals
- Real RTCM3 from base station
- Real LAMBDA ambiguity resolution
- True centimetre accuracy
- Full 5-module integration launch
- **RTCM: Full hardware chain**

RTCM Implementation Progress Across Levels

RTCM Component	Level 1	Level 2	Level 3	Level 4
RTK state machine	✓ Timer	✓ Quality score	✓ Quality score	✓ Hardware
Gaussian noise model	✓	✓	✓	Replaced by real
RTCM3 binary generation	✗	✗	⟶ pyrtcm	✓ Real receiver
CRC-24Q checksums	✗	✗	⟶ pyrtcm	✓

RTCM Component	Level 1	Level 2	Level 3	Level 4
GPS_RTCM_DATA MAVLink	✗	✗	⟶ pymavlink	✓
MAVLink fragmentation	✗	✗	⟶ pymavlink	✓
PX4 RTCM reassembly	✗	✗	⟶ PX4 SITL	✓
GPS UART injection	✗	✗	✗ (Gazebo limit)	✓ u-blox F9P
LAMBDA algorithm	✗	✗	✗	✓ u-blox firmware
Real centimetre accuracy	✗ (modelled)	✗ (modelled)	✗ (modelled)	✓ Physical

Table 7.1 — RTCM component implementation status across all four levels

Simulation Progress Bar

What Does Not Change When Moving to Real Hardware

The hardware-agnostic design: The core RTK logic — `gnss_noise_model.py`, `rtk_correction_model.py`, `coordinate_transform.py`, `rtk_status_manager.py` — does not need to change at any level, including Level 4. The ROS 2 topic interfaces (`/uav/rtk_position`, `/uav/rtk_status`, `/uav/raw_gps`) remain identical. The only swap is the adapter node: `rtcm_mavlink_bridge_node.py` (software pyrtcm source) → real hardware driver node (u-blox serial driver). Everything else is unchanged.

Key Takeaways for Supervisor Communication

Point	What to Say
What L1 & L2 proved	

Point	What to Say
	The RTK architecture, ROS 2 topic interfaces, coordinate transforms, noise model, and correction-loss recovery all work correctly. 96.7% accuracy improvement demonstrated.
Honest RTCM limitation	RTCM correction data was abstracted as a quality score float, not real binary. The MAVLink GPS_RTCM_DATA protocol was not implemented in L1 or L2.
What L3 adds	Real RTCM3 binary frames generated by pyrtcm, transmitted via GPS_RTCM_DATA (ID 233) using pymavlink. PX4 receives them. The full communication protocol is proven — no hardware required.
Why L3 doesn't change accuracy	Gazebo's GPS plugin cannot process RTCM corrections — it has no parser, no carrier-phase model. Accuracy remains driven by the noise model. This is an inherent SITL limitation, not a code error.
Path to hardware	Level 4 connects a real u-blox F9P base station. The rtcm_mavlink_bridge_node is replaced by a real hardware driver. Everything else — the ROS 2 architecture built across L1–L3 — is production-ready unchanged.

Table 7.2 — Summary points for supervisor communication

FINAL STATEMENT: Level 3's claim is precise and defensible — "The RTCM correction protocol is implemented at the MAVLink communication layer. Real RTCM3 binary frames are transmitted to PX4 via GPS_RTCM_DATA (ID 233) using the correct fragmentation protocol. When a real u-blox RTK receiver is connected, only the RTCM source node changes. The entire stack from MAVLink bridge to ROS 2 topic output is hardware-agnostic and production-ready."